
CS 301

High-Performance Computing

Lab Report-06

Parallel Interpolation

Performance Analysis

Soham Mevada (202301484)
Atik Vohra (202301447)

Contents

1	Introduction	3
2	Hardware Details	3
2.1	LAB207 PCs	3
2.2	HPC Cluster	3
3	Answers to Analysis and Questions in Lab 6	3
3.1	Pseudocode and Diagram	3
3.2	Parallel Implementation and Race Conditions	5
3.3	Speedup and Execution Time Graphs	6
3.4	Parallel Efficiency and scalability	9
3.5	Serial vs Parallel Performance	9
3.6	Explanation of Observations	9
3.7	Optimization with Unlimited Resources	10
3.8	Load Imbalance and Bottlenecks	10
3.9	Alternative Approaches	10
3.10	Advanced Optimization Ideas	10
4	Conclusion	11

1 Introduction

2 Hardware Details

2.1 LAB207 PCs

- Architecture: x86_64
- CPU(s): 12
- Threads per core: 2
- Cores per socket: 6
- Processor: Intel Core i5-12500
- L1 Cache: 288K
- L2 Cache: 7.5M
- L3 Cache: 18M

2.2 HPC Cluster

- Architecture: x86_64
- CPU(s): 16
- Threads per core: 1
- Cores per socket: 8
- Sockets: 2
- Processor: Intel Xeon E5-2640 v3
- L1 Cache: 32K
- L2 Cache: 256K
- L3 Cache: 20480K

3 Answers to Analysis and Questions in Lab 6

3.1 Pseudocode and Diagram

Pseudocode

Input: `particles[]`, `global mesh_value[]`, `thread-local mesh pad`

Initialize:

`np = number of particles`

```

nt = number of threads
ms = grid size (gx * gy)

Parallel Region (OpenMP with nt threads):
    Each thread gets its own local mesh: t_mesh

Parallel for each particle p:
    Read particle position (px, py)

    Compute grid indices (i, j)
    Compute distances within cell (lx, ly)

    Compute bilinear interpolation weights:
        w00, w10, w01, w11

    Compute base index in grid

    Accumulate weights into thread-local mesh:
        t_mesh[base]      += w00
        t_mesh[base + 1]  += w10
        t_mesh[base + gx] += w01
        t_mesh[base + gx+1] += w11

Synchronization phase:
    For each section of mesh we distribute among threads.
    Then we add all the local thread values into the original mesh.

Output: Updated global mesh_value

```

Illustrative Diagram

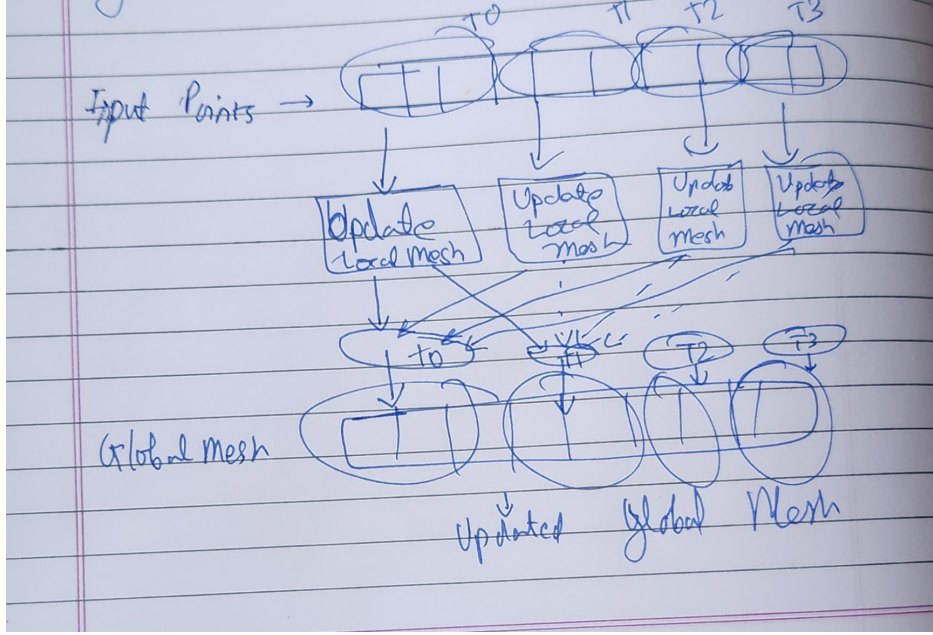


Figure 1: Parallel interpolation workflow showing particle distribution, thread-local accumulation, and final merge to global mesh

3.2 Parallel Implementation and Race Conditions

Parallel Approach:

The interpolation algorithm is parallelized using OpenMP by distributing particles across multiple threads. Each thread processes a subset of particles independently using a `#pragma omp for` loop with static scheduling.

Instead of directly updating the global mesh, each thread maintains a private copy of the mesh (`t_mesh`) stored inside a preallocated structure (`LocalMeshPad`). This ensures that all particle-to-grid interpolation computations are performed without interference from other threads.

The overall workflow is:

- Distribute particles across threads
- Perform interpolation independently using thread-local meshes
- Merge results into the global mesh after computation

Handling Race Conditions:

The primary race condition arises when multiple threads attempt to update the same grid cell simultaneously. This is avoided using a two-phase strategy:

1. **Thread-Local Accumulation:** Each thread writes only to its own local mesh (`t_mesh`), eliminating conflicts during interpolation.

2. **Atomic Merge Phase:** After computation, all threads contribute their local mesh values to the global mesh using: mesh sections divided across the threads and each thread updates global meshes from the local meshes. This ensures correctness while minimizing synchronization overhead.

Why this works:

- Eliminates fine-grained contention during interpolation
- Reduces the need for expensive critical sections
- Ensures correctness with minimal synchronization by dividing work across threads

Trade-offs:

- Increased memory usage due to per-thread meshes

3.3 Speedup and Execution Time Graphs

Using Generated Plots

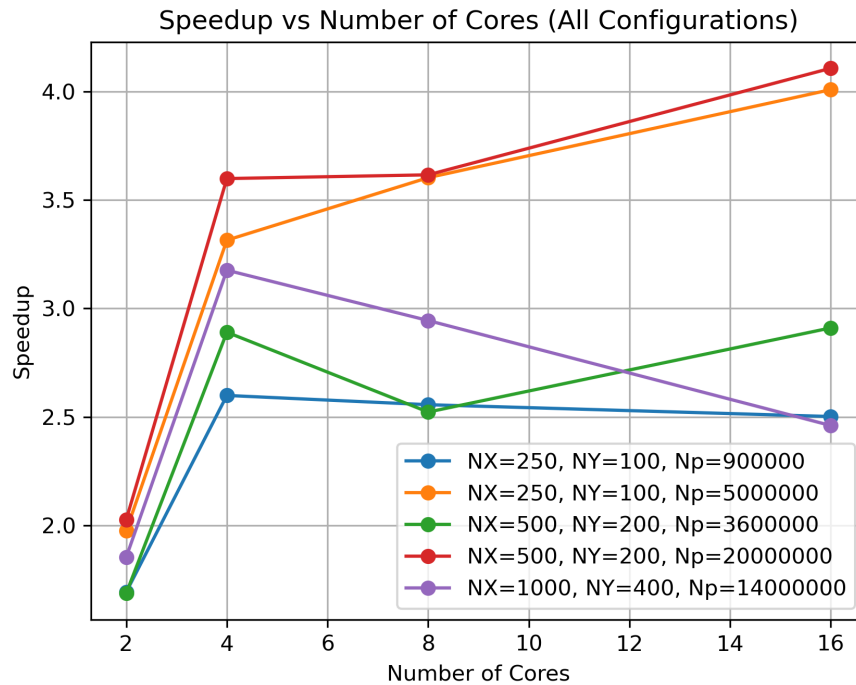


Figure 2: Speedup vs Cores (LAB PC)

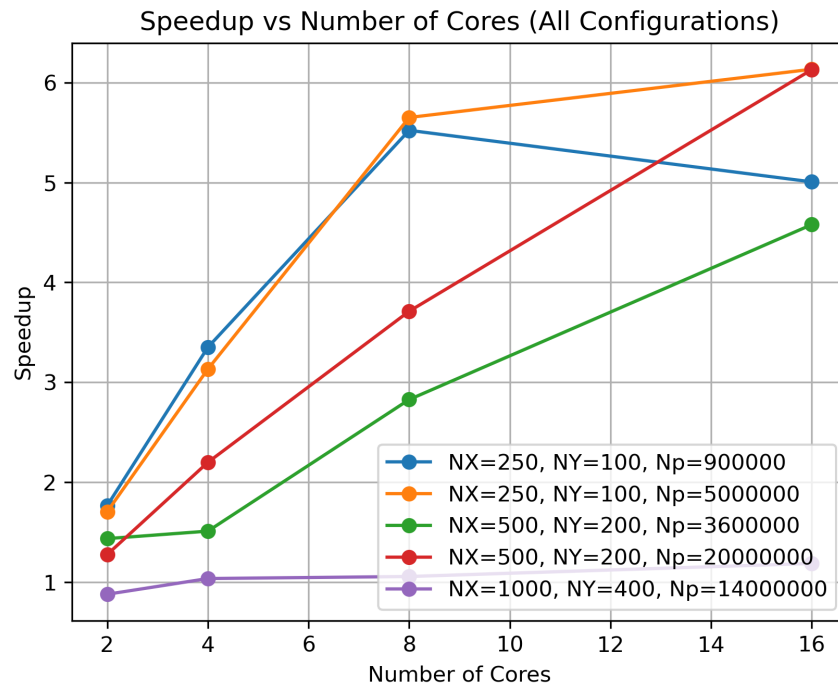


Figure 3: Speedup vs Cores (CLUSTER)

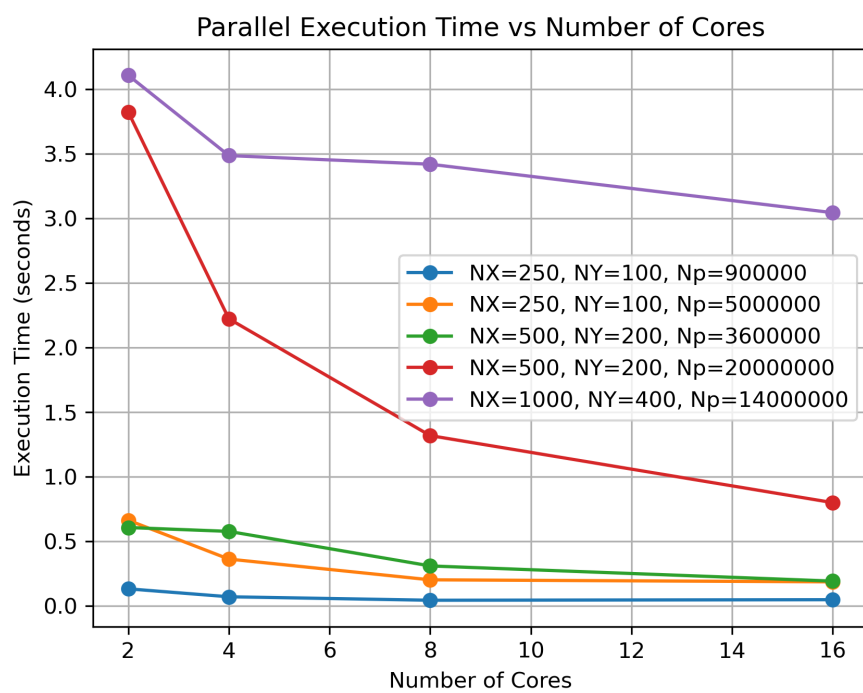


Figure 4: Execution Time vs Cores (LAB PC)

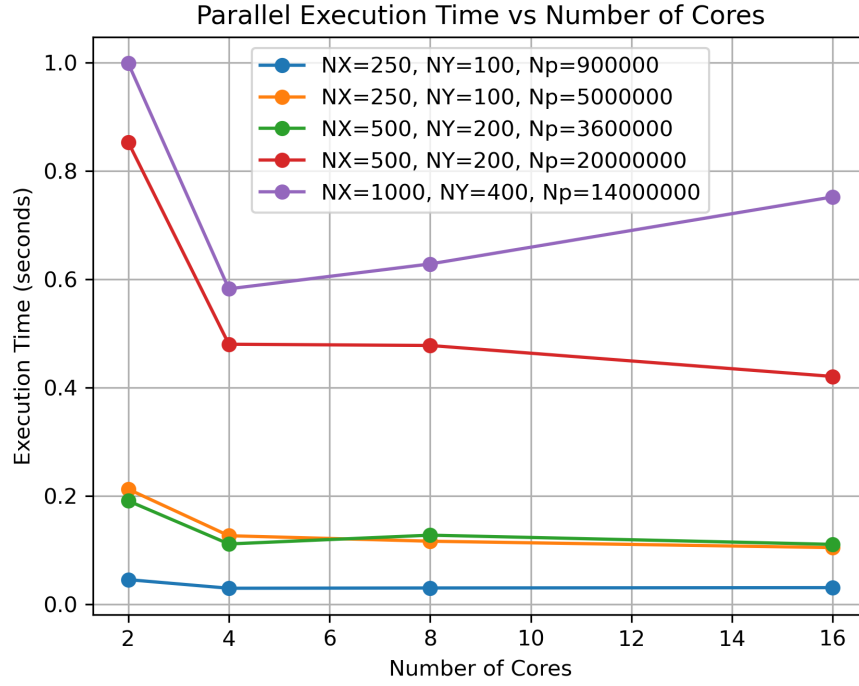


Figure 5: Execution Time vs Cores (CLUSTER)

3.4 Parallel Efficiency and scalability

Observation:

The parallel efficiency shows a decreasing trend as the number of cores increases, dropping from 63.9% at 2 cores to 38.3% at 16 cores. This indicates moderate scalability.

The speedup increases with core count, reaching a maximum of 6.128 at 16 cores, which demonstrates effective parallelization but still falls short of ideal linear scaling.

3.5 Serial vs Parallel Performance

Maximum Speedup Achieved:

6.128 (Cluster)

Comparison:

- Serial execution time: 4.89 seconds
- Best parallel time: 0.798 seconds

3.6 Explanation of Observations

The observed speedup behavior can be explained using fundamental HPC principles:

- **Amdahl's Law:** The presence of a serial portion (such as the merge phase) limits the maximum achievable speedup, preventing ideal linear scaling.

- **Memory Bandwidth Limits:** The interpolation involves irregular memory accesses (scatter operations), making the implementation memory-bound. As cores increase, memory bandwidth becomes a bottleneck.
- **Cache Effects:** At lower core counts, better cache utilization improves performance. However, at higher cores, cache contention and misses increase, reducing efficiency.

3.7 Optimization with Unlimited Resources

With unlimited HPC resources, the implementation can be further optimized using:

- GPU acceleration (CUDA) for massive parallelism
- Distributed computing using MPI for large-scale simulations
- Domain decomposition to divide the grid across nodes
- Hybrid MPI + OpenMP approach for multi-node and multi-core scalability

3.8 Load Imbalance and Bottlenecks

The following bottlenecks were observed:

- **Memory contention:** Multiple threads accessing nearby memory locations reduces bandwidth efficiency.
- **Load imbalance:** Although static scheduling is used, uneven particle distribution across grid cells can lead to imbalance in workload.

3.9 Alternative Approaches

The performance can be improved using:

- Cell-linked list to localize particle-grid interactions
- Spatial hashing to reduce memory access overhead
- Structure of Arrays (SoA) to improve cache efficiency and enable vectorization

3.10 Advanced Optimization Ideas

Further performance improvements can be achieved using:

- Lock-free or reduction-based merging instead of atomic operations
- Optimized OpenMP scheduling (e.g., dynamic or guided)
- SIMD vectorization to exploit data-level parallelism
- Cache blocking techniques to improve memory locality

4 Conclusion

In this lab, we implemented and analyzed a parallel interpolation algorithm using OpenMP. The use of thread-local meshes helped reduce race conditions and enabled effective parallel execution.

The results show moderate scalability, with speedup increasing up to 6.128 at 16 cores. However, efficiency decreases with higher core counts due to synchronization overhead and memory bandwidth limitations, indicating a memory-bound implementation.

Overall, the study highlights the importance of minimizing synchronization and optimizing memory access patterns to achieve better parallel performance.